Files hidden in shared chats

we are making a hrms : recruitment and onboarding, and core hr. we need to proceed for our backend and were planning to use php/laravel and also we are planning to integrate nlp ner for our applicant screening and OCR. we might use python. can u suggest a list on how we are starting this project? like approaches, structures, what needs to setup, everything.

can u send a step by step on how to do it?

we already made the frontend.

This is a solid plan to nail down before writing code. Here's how I'd sequence it: a quick look at the overall architecture, then a full step-by-step guide with folder structures you can copy directly.

Interactive visual hidden in shared chats

The flow: applicant uploads a resume → Laravel stores it and calls the Python service → spaCy NER + OCR pull out entities → a score comes back and gets saved. Laravel and MySQL stay the system of record; Python stays a stateless "smart" service it calls into.

Here's the full step-by-step guide, with folder structures for all three devs to work from.

Created a file, read a file

Quick summary of what's in it:

- **Architecture**: Laravel is the source of truth, Python NLP service is stateless and only ever called by Laravel (not the frontend) — keeps auth in one place and lets you rewrite the ML part later without touching React.
- **Laravel folder structure**: modular, one folder per subsystem (`Modules/ApplicantManagement`, `Modules/CoreHCM`, etc.) using `nwidart/laravel-modules`, so each dev owns a folder with minimal merge conflicts.

- **Python service folder structure**: FastAPI app split into `ocr_service`, `ner_service`, `scoring_service`, with a defined API contract Laravel calls.
- **Step-by-step setup**: Laravel init → auth/roles → module scaffolding → Python venv/spaCy/Tesseract → connecting the two → Docker Compose → frontend cleanup (removing Lovable-specific bits like the Supabase client and the `lovable-tagger` plugin) → git workflow.
- **3-dev split** mapped to your subsystems, plus a recommended **build order** so nobody's blocked waiting on someone else's module.

Want me to generate the actual `docker-compose.yml`, the Laravel `NlpService.php` HTTP client, or the FastAPI skeleton next?

Hrms backend setup guide
Document · MD

Download

how to install nwidart? im gonna type this in the terminal?

Yes — run this inside your `backend-laravel` folder:

```bash
composer require nwidart/laravel-modules
```

That's the only command needed to install it. A few notes:

1. **Run it from the Laravel root** — the folder that has `artisan`, `composer.json`, etc. Based on your terminal screenshot, that'd be something like:

```bash
cd "C:\COLLEGE\4TH YEAR\final repo\Hotel-and-Restaurant-HR1\backend-laravel"
composer require nwidart/laravel-modules
```

2. **Check your Laravel version first** — newer `nwidart/laravel-modules` versions require Laravel 10/11+. If `composer require` throws a version conflict, check the package's GitHub releases for the version matching your Laravel, and install a specific version like:

```bash
composer require nwidart/laravel-modules:^11.0
```

3. **Publish the config (optional but useful)** — lets you customize where modules live, namespace, etc.:

```bash
php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider"
```

4. **Verify it worked:**

```bash
php artisan module:make TestModule
```

If it creates a `Modules/TestModule` folder, it's working — delete that test folder after.

5. Then create your real modules:

```bash
php artisan module:make ApplicantManagement RecruitmentManagement NewHireOnboarding Co
```

(you can pass multiple names to `module:make` in one line, no need to run it 8 times)

One thing to confirm — do you have Composer installed and working already (i.e. does `composer --version` return something)?

ⓘ Turn on web search in Search and tools menu. Otherwise, links provided may not be accurate or up to date.

i made folders : frontend/ # existing React app (from Lovable export) ├── backend-laravel/ # Laravel API ├── nlp-service/ # Python FastAPI + spaCy + OCR ├── docs/

Good — now let's actually populate them. Since you already created the folders (rather than letting Composer create `backend-laravel` for you), you need to install *into* the existing empty folder.

Move your existing frontend in

Move your Lovable-exported React project's contents into the frontend/ folder you made (package.json, src/, index.html, vite.config.ts, etc. should sit directly inside frontend/, not nested another level deep).

1 2 3 4 5 6 View all steps Next

One important note on step 2: `composer create-project laravel/laravel .` only works cleanly if `backend-laravel/` is completely empty — if there's even a stray `.gitkeep` or `.git` folder in there, Composer may refuse. If that happens, just delete the folder and let Composer recreate it:

bash

```
cd ..
rmdir backend-laravel
composer create-project laravel/laravel backend-laravel
```

Do you want me to also generate the actual `docker-compose.yml` now so all three services (plus MySQL) can run with one command? That'll save you from installing PHP/Composer/Python locally for every teammate.

what will be the role of docker? is this necessary? were also considering for our hosting/domain for deployment

Docker's role here is **consistency and packaging** — it's not the only way to get there, so let's break down what it actually buys you.

What Docker does for you:

1. **"Works on my machine" prevention** — right now, dev A might have PHP 8.2, dev B has 8.3, one has a different MySQL version, one forgot to install Tesseract for OCR. Docker locks all of that into config files everyone shares, so `docker compose up` gives identical environments to all 3 of you and to your deployment server.
2. **One-command startup** — instead of each dev manually starting MySQL, `php artisan serve`, and `uvicorn` separately, one command boots everything.
3. **Deployment parity** — if the exact same Docker setup that runs on your laptop runs on the server, you eliminate a whole class of "worked locally, broke in production" bugs — especially relevant here since you have a *Python service with system-level dependencies* (Tesseract OCR binary, spaCy models) that's easy to misconfigure on a shared host.

Is it necessary? No — plenty of capstone projects ship without it. You *can* run Laravel via XAMPP/Laragon, Python via a plain venv, and deploy to a shared host or VPS manually.

Docker adds a learning curve and another thing to debug if you're not already comfortable with it.

Where it matters more for your specific case: your NLP service depends on Tesseract (OS-level binary) and possibly a custom spaCy model — those are exactly the kind of dependencies that are painless in Docker and painful on shared/cPanel-style hosting, which often can't install system binaries at all.

On hosting/domain — this is the deciding factor. What are you leaning toward?

is this structute :

```
backend-laravel/
├── app/
│   ├── Http/Middleware/       # shared middleware (role checks, audit logging)
│   ├── Models/                # ONLY truly shared models (User, Role)
│   └── Services/
│       └── NlpService.php     # single HTTP client wrapper for calling Python
├── Modules/
│   ├── ApplicantManagement/
│   │   ├── Http/Controllers/
│   │   ├── Http/Requests/
│   │   ├── Models/            # Applicant, Resume, ApplicantDocument
│   │   ├── Database/Migrations/
│   │   ├── Routes/api.php
│   │   ├── Services/          # calls App\Services\NlpService
│   │   └── Resources/         # API resource/transformers
│   ├── RecruitmentManagement/
│   │   ├── Models/            # JobPosting, JobRequisition, Interview
│   │   └── ... (same pattern)
│   ├── NewHireOnboarding/
│   │   ├── Models/            # OnboardingChecklist, OnboardingTask
│   │   └── ...
│   ├── CoreHCM/
│   │   ├── Models/            # Employee, Department, Position
│   │   └── ...
│   ├── EmployeeSelfService/
│   │   ├── Models/            # LeaveRequest (read-only views into other modules),
Requests
│   │   └── ...
│   ├── EmployeeRecords/
│   │   ├── Models/            # EmployeeDocument, EmploymentHistory
│   │   └── ...
│   ├── UserManagement/
│   │   ├── Models/            # User, Role, Permission (Spatie)
│   │   └── ...
│   └── AuditLog/
│       ├── Models/            # ActivityLog
```

```
│   └── ...
├── routes/
│   └── api.php            # only global/public routes (auth, jobs listing)
├── database/
│   └── migrations/        # only truly shared tables (users, roles)
├── config/
├── .env.example
└── composer.json
```

separated with python? is this structure solely for php laravel?

Show more

Yes — that whole tree is **100% inside `backend-laravel/`, PHP/Laravel only.** Python lives completely separately in its own `nlp-service/` folder, with zero shared files between them. They only talk over HTTP, like two independent apps.

To be precise about the boundary:

```
hotel-restaurant-hrms/
├── backend-laravel/        ← the tree you pasted, PHP only
│   ├── app/
│   ├── Modules/
│   ├── routes/
│   └── ...
│
├── nlp-service/             ← completely separate, Python only
│   ├── app/
│   │   ├── main.py
│   │   ├── api/routes/
│   │   ├── services/        ← ocr_service.py, ner_service.py (spaCy lives here)
│   │   └── schemas/
│   ├── models_spacy/
│   ├── requirements.txt
│   └── venv/
│
└── frontend/                ← React, also separate
```

The only "connection point" between Laravel and Python is that one file: `app/Services/NlpService.php`. That file doesn't contain any Python code — it just makes an HTTP request (`Http::post(...)`) to wherever `nlp-service` is running (e.g. `http://localhost:8001/screening/score`) and gets JSON back. That's the entire integration surface.

A couple of clarifications on the tree itself, since you're about to build it:

- `app/Models/` and `Modules/*/Models/` — you don't create these folders manually. Once `nwidart/laravel-modules` is installed, `php artisan module:make ApplicantManagement` auto-generates the `Http/Controllers`, `Models`, `Database/Migrations`, `Routes` subfolders inside `Modules/ApplicantManagement/` for you. The `# comments` in the tree are just notes on *what goes there*, not folders you make by hand.
- `app/Services/NlpService.php` — this one you do create manually, it's not scaffolded by anything.
- `routes/api.php` at the root only holds things that don't belong to a specific module (login, public job listing landing route), everything else routes through each module's own `Routes/api.php`.

Want me to write the actual `NlpService.php` file so you have the real connector in hand before your team starts building modules against it?